

# 디자인-코드 핸드오프 워크플로우

디자이너, 개발자, 에이전트가 Figma 디자인을 코드로 옮기기 위한 협업 흐름이다. 4-Phase 구조 위에서 반응형 디자인과 shadcn/ui 사용을 1급으로 다루며, 각 단계에서 디자이너와 개발자가 정확히 무엇을 하는지, 스킬이 어떤 입력을 받아 어떤 결과를 내는지를 명확히 기술한다.

이 워크플로우는 세 스킬을 순서대로 연결한다.

- `design-md`: 디자인 시스템과 token artifact를 만든다. 반응형 breakpoint 위치와 shadcn token 정렬을 다룬다.
- `components`: Figma component와 code component의 mapping contract를 관리한다. shadcn 감지/origin/custom을 다룬다.
- `figma-implement`: 완성된 Figma component 또는 page frame을 UI code로 구현한다. 다중 프레임 반응형 병합과 shadcn 재사용/설치를 다룬다.

스킬별 상세 설명:

| 스킬                           | 설명 문서                                       |
|------------------------------|---------------------------------------------|
| <code>design-md</code>       | <code>docs/skills/design-md.md</code>       |
| <code>components</code>      | <code>docs/skills/components.md</code>      |
| <code>figma-implement</code> | <code>docs/skills/figma-implement.md</code> |

## 역할 한눈에 보기

| 주체   | 핵심 책임                                                                                                                                              |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| 디자이너 | Figma에서 디자인 시스템·컴포넌트·페이지를 만든다. token을 사용하고, breakpoint별 프레임을 그리고, shadcn Kit이면 이름·variant를 정렬한다.                                                   |
| 개발자  | 스킬을 실행(에이전트에 지시)하고, 산출물을 코드베이스에 통합한다. token import, <code>--breakpoint-*</code> /shadcn 변수 wiring, shadcn 설치, <code>COMPONENTS.md</code> 검토를 담당한다. |
| 에이전트 | Figma MCP로 디자인을 읽어 token artifact· <code>COMPONENTS.md</code> ·UI code를 생성/갱신한다. 추측 대신 <code>COMPONENTS.md</code> 와 token을 따른다.                    |

## 전제 조건

- Figma 파일은 가능하면 2개로 분리한다.
  - File A / File 1:** 디자인 시스템, foundations, variables, styles, component library
  - File B / File 2:** 실제 제품 페이지, File A library component instance로 구성
- 디자이너는 File 1 component를 library에 publish한다.

- File 2의 제품 페이지는 detach된 raw frame이 아니라 File 1 component instance를 우선 사용한다. (반응형 인식의 전제조건 — "횡단 규칙 1" 참조)
- 에이전트는 Figma MCP에 접근할 수 있어야 한다.
- 앱은 Tailwind v4 기준으로 `tailwind.theme.css`를 import할 수 있어야 한다.
- 별도 자동 매핑 기능은 사용하지 않는다. Figma component와 code component의 공식 매핑 문서는 `COMPONENTS.md`다.
- 반응형 프로젝트라면 breakpoint 값을 `DESIGN.md`의 `## Layout`에 문서화하고, 커스텀 breakpoint는 앱의 `globals.css`에 `--breakpoint-*`로 둔다. (`tailwind.theme.css`에는 넣지 않는다 — "횡단 규칙 1" 참조)
- shadcn/ui를 쓰면 프로젝트 루트에 `components.json`이 있고, shadcn primitive는 `components/ui`에 설치된다. shadcn은 외부 라이브러리가 아니라 프로젝트의 primitive로 취급한다.

## 전체 흐름

### Phase 1. design-md

File 1 foundations/design system 분석

-> DESIGN.md, tailwind.theme.css, tokens.studio.global.json

-> handoff mode라면 COMPONENTS.md 초안도 생성

-> Layout 섹션에 breakpoint 문서화, shadcn token 정렬 매핑 보고

### Phase 2. components

File 1 component library 분석 + 코드베이스 shadcn 감지

-> COMPONENTS.md mapping table 누적 갱신 (Origin: shadcn / shadcn(customized) / custom)

### Phase 3. 제품 페이지 디자인

디자이너가 File 2에서 File 1 component instance로 page frame 완성

-> 반응형이면 breakpoint별 프레임을 같은 instance로 그래서 node-id URL들을 전달

### Phase 4. figma-implement

File A component 또는 File B frame(들)과 handoff artifacts 소비

-> component-level 또는 page-level code 구현

-> 다중 프레임 mobile-first 병합, shadcn 재사용/설치

## 횡단 규칙 1 — 반응형

반응형은 "컴포넌트 내부 변형"과 "페이지 레이아웃 재배치" 두 종류이며, 사는 곳이 다르다.

| 반응형 종류             | 정의                                   | 사는 곳              | 표현 방법                                                           |
|--------------------|--------------------------------------|-------------------|-----------------------------------------------------------------|
| <b>variant 반응형</b> | <code>size=sm/md/lg</code> 같은 이산적 변형 | File 1 컴포넌트       | Figma variant → 코드 props                                        |
| <b>layout 반응형</b>  | desktop 3열 → mobile 1열 stack 등 재배치   | File 2 page frame | breakpoint별 프레임 + 코드의 <code>md: 1</code><br><code>g:</code> 접두사 |

핵심 사실들:

1. **breakpoint는 DESIGN.md 토큰이 아니다.** design.md spec에는 breakpoint token group이 없다. breakpoint 값은 `DESIGN.md`의 `## Layout` prose에 명시한다(예: `sm 640 / md 768 / lg 1024 / xl 1280`). 가능하면 Tailwind 기본값과 맞춘다.
2. **커스텀 breakpoint는 `tailwind.theme.css`가 아니라 `globals.css`에 둔다.** `tailwind.theme.css`는 export 산출물이라 "fresh export와 일치" 검증을 받는데, exporter는 `--breakpoint-*`를 만들지 않는다. 따라서 커스텀 breakpoint는 앱의 `globals.css`에 `--breakpoint-*`로 선언한다.
3. **에이전트는 여러 프레임이 "같은 화면의 반응형 변형"임을 자동으로 알지 못한다.** 한 프레임 내부의 Auto Layout(flex-direction/gap/sizing)은 안정적으로 읽지만, 프레임 간 병합은 다음 3조건이 모두 필요하다.
  - (a) **같은 File 1 instance 사용** — 노드 정체성 앵커. instance를 detach하면 깨진다.
  - (b) **breakpoint별 프레임을 함께 전달** — desktop 하나만 주면 desktop 전용으로 박힌다.
  - (c) **"병합하라" 지시** — 안 주면 별개 컴포넌트 3개로 갈라질 수 있다.
  - 조건이 갖춰지면 에이전트가 프레임을 diff하여 `flex-col lg:flex-row`, `w-full lg:w-[1024px]` 식으로 mobile-first 병합한다.

디자이너가 할 일: breakpoint별 프레임을 **같은 instance**로 그리고, 화면만으로 표현 안 되는 동작은 Notes로 적는다. 개발자가 할 일: breakpoint 값을 `globals.css`에 wiring하고, figma-implement에 프레임들을 함께 주며 mobile-first 병합을 지시한다.

## 횡단 규칙 2 — shadcn/ui

shadcn은 라이브러리 import가 아니라 코드를 `components/ui`에 복사해 넣는 방식이다. `COMPONENTS.md`의 Lookup Order 2순위가 정확히 `components/ui`이므로, shadcn을 프로젝트 **primitive**로 취급하면 기존 흐름에 그대로 들어온다.

1. **감지** — 프로젝트 루트의 `components.json`이 가장 강한 신호. `components/ui`의 kebab-case 파일 + `cn()` / `cva` 패턴도 신호.
2. **origin 분류** — `COMPONENTS.md`에 각 컴포넌트의 출처를 기록한다.
  - `shadcn`: 수정 안 한 표준 shadcn. `npm shadcn add <name>`로 재생성 가능.
  - `shadcn (customized)`: 수정한 shadcn. `do not regenerate` 플래그 + 바뀐 variant/props를 반드시 기록. (재설치 시 커스텀 소실 방지)
  - `custom`: 팀이 만든 컴포넌트.
3. **미설치 표준 shadcn** — 손으로 짜지 말고 `npm shadcn@latest add <name>`로 설치 후 등록. (`COMPONENTS.md`에 `pending` → 실제 경로)

4. **Figma Kit** — 디자이너가 공식 shadcn Figma Kit을 쓰면 이름·variant가 shadcn props API와 맞아 매핑이 깔끔해진다. 다른 `Variants/Notes`에 매핑 note.
5. **token 정렬** — shadcn은 `--primary`, `--background`, `--radius` 등 CSS 변수를 읽는다. `design-m`가 DESIGN.md token ↔ shadcn 변수 매핑을 보고하고, 개발자가 `globals.css`에 wiring한다.

shadcn인지 custom인지 구분 방법: Figma의 **library 출처**, **컴포넌트 이름**(shadcn 표준 집합과 일치하는지), **variant/props** 구조. 이 판단을 `COMPONENTS.md`의 `Origin`에 한 번 기록하면 이후 에이전트가 매번 추론하지 않는다.

## Phase 1 - 디자인 시스템 수립

*트리거: 프로젝트 시작 또는 색상, 타이포그래피, spacing, radius, breakpoint 등 foundation 변경 시*  
**스킬:** `design-md` **입력:** File 1의 Foundations 또는 Design System 페이지 URL

### 디자이너가 하는 일

- File 1에 foundations(색/타이포/spacing/radius)와 breakpoint 정책을 정리한다.
- shadcn Kit을 쓸 경우 Kit을 File 1 library 기반으로 활성화한다.

### 개발자 -> 에이전트 (스킬 실행)

```
File 1 Foundations URL: <url>
design-md 스킬 실행.
DESIGN.md, tailwind.theme.css, tokens.studio.global.json을 생성해줘.
반응형이면 breakpoint 값을 Layout 섹션에 문서화하고,
shadcn을 쓰면 DESIGN.md token ↔ shadcn 변수 매핑을 보고해줘.
```

핸드오프 패키지가 필요하면 `COMPONENTS.md` 초안까지 요청한다.

### 에이전트 처리 흐름

1. `references/DESIGN_SPEC.md`를 읽는다.
2. Figma 증거 수집 + 프로젝트 컨텍스트 감지(`components.json`, breakpoint 셋업).
3. `DESIGN.md` 작성(반응형이면 `## Layout`에 breakpoint/reflow 문서화).
4. `lint` → 통과 → `css-tailwind` export → `tokens.studio.global.json` 생성 → 검증.
5. (handoff) `COMPONENTS.md` 초안 생성.

## 에이전트 산출물

| 파일                        | 소비자       | 역할                                                        |
|---------------------------|-----------|-----------------------------------------------------------|
| DESIGN.md                 | 에이전트, 개발자 | 디자인 시스템 source of truth (breakpoint는 Layout prose)        |
| tailwind.theme.css        | 개발자, 앱    | Tailwind v4 @theme token CSS (breakpoint/shadcn 변수는 미 포함) |
| tokens.studio.global.json | 디자이너      | Tokens Studio global token-set JSON                       |
| COMPONENTS.md             | 에이전트, 개발자 | handoff mode component contract 초안                        |

## 개발자 검토/통합

- tailwind.theme.css 를 앱의 Tailwind entry stylesheet에 import한다.
- 커스텀 breakpoint를 globals.css 에 --breakpoint-\* 로 선언한다.
- shadcn 사용 시 보고된 매핑대로 globals.css 에 shadcn 변수(--primary 등)를 token 값으로 wiring한다.
- 디자이너는 tokens.studio.global.json 을 Tokens Studio global token set JSON View에 붙여 넣어 확인한다.

## 주의점

- DESIGN.md 의 ## Components 에는 reuse policy만 둔다. 실제 path/mapping/variant는 COMPONENT S.md .
- breakpoint를 breakpoints: 같은 top-level token group으로 넣지 않는다(spec 밖).

## Phase 2 - 컴포넌트 계약 수립

**트리거:** Figma component 추가/수정, variant/property 변경, code path 확정, shadcn 컴포넌트 설치

**시 스크립:** components    **입력:** File 1의 개별 component URL 또는 Components page URL (+ 선택: 코드베이스 경로)

Phase 2는 반복 루프다.

디자이너: Figma component 제작/publish (shadcn Kit이면 이름 ·variant 정렬)

↓

개발자: components 스킴 실행

↓

에이전트: COMPONENTS.md 추가/갱신 (Origin 분류 + shadcn 감지)

↓

개발자 또는 에이전트: figma-implement로 pending code component 구현/설치

↓

COMPONENTS.md의 pending path를 실제 경로로 갱신

## 디자이너가 하는 일

- Phase 1 token을 사용한다(raw 값 금지).
- variant/property 이름을 code props와 맞춘다. shadcn Kit이면 shadcn props API(`variant=default`  
`t/...`, `size=default/sm/lg/icon`)와 정렬.
- publish하고, 제품 페이지에서 instance를 detach하지 않는다.

## 개발자 -> 에이전트 (스킴 실행)

File 1 component URL 또는 Components page URL: <url>

components 스킴 실행.

COMPONENTS.md를 갱신해줘. 기존 entry는 삭제하지 말고,

코드 경로 미확인이면 pending으로 뒀.

shadcn 컴포넌트는 Origin을 표시하고, 커스텀된 건 'do not regenerate'로 표기해줘.

## 에이전트 처리 흐름

1. `COMPONENTS.md` 존재 여부 확인.
2. shadcn 감지(`components.json`, `components/ui` 스캔).
3. Figma MCP로 component structure/variants 수집.
4. 코드 스캔으로 경로 확정 + origin 분류(`shadcn` / `shadcn (customized)` / `custom`).
5. entry 추가/갱신(기존 삭제 금지). 커스텀 shadcn은 `do not regenerate` + 변경 variant 기록.
6. 이름 불일치 시 `Variants/Notes`에 매핑 note.

## COMPONENTS.md 권장 구조

```
# Component Contract

## Lookup Order
1. COMPONENTS.md mapped Code Path
2. components/ui
3. components/sections
4. route-local components
5. new component - reason required

## Components
| Figma Name | Code Path | Origin | Variants/Notes |
|-----|-----|-----|-----|
| Button | components/ui/button.tsx | shadcn | variant=default/destructive/outline/ghost, size=sm/default/lg |
| Badge | components/ui/badge.tsx | shadcn (customized) | do not regenerate; 커스텀 variant=brand 추가 |
| PricingCard | components/sections/PricingCard.tsx | custom | variant=monthly/annual |

## Primitive Components
| Code Path | Origin | Role | Notes |
|-----|-----|-----|-----|
| components/ui/button.tsx | shadcn | button primitive | installable via `npx shadcn add button` |

## New Component Rule
New shared components are allowed only when existing components cannot express the required UI through props, variants, slots, or composition. Every new component must include the reason existing components are insufficient.
```

**Origin** 컬럼이 핵심이다. 3컬럼 형식을 유지해야 하면 같은 정보를 **Variants/Notes** 앞머리에 둔다.

## 주의점

- Figma 이름, **COMPONENTS.md** entry 이름, code 이름을 일치시킨다.
- **pending** 은 실제 미구현/미확인(미설치 표준 shadcn 포함)에만.
- 기존 mapping은 의도 없이 삭제하지 않는다.

## Phase 3 - 제품 페이지 디자인

**트리거:** 구현할 기능 또는 페이지가 확정되었을 때 **담당:** 디자이너 **입력:** File 1 library, Phase 2의 component contract

### 디자이너가 하는 일

- File 2에서 File 1 library를 활성화하고, page를 File 1 component instance 중심으로 구성한다.
- 화면별 state를 포함한다.
- **반응형이면:** breakpoint별(mobile/tablet/desktop) 프레임을 **같은 instance**를 재사용해서 그린다. detach하지 않는다(반응형 인식의 정체성 앵커).
- frame만으로 표현 안 되는 reflow 규칙(예: "lg 3열 → md 2열 → 그 이하 1열 stack", "mobile에서 sidebar→drawer")은 Notes에 글로 적는다.
- 구현 가능한 수준으로 spacing/wrapping/asset crop을 정리한다.

### 전달물

단일 화면:

```
Figma URL: <File 2 frame URL with node-id>
Target route: <route-or-file-path>
Notes: <interaction/state/asset 설명>
```

반응형 화면:

```
Figma URLs (같은 화면, 같은 instance, breakpoint별):
mobile   : <node-id URL>
tablet   : <node-id URL>
desktop  : <node-id URL>
Target route: <route-or-file-path>
Notes: <breakpoint별 reflow 규칙 + interaction/state>
```

## Phase 4 - 구현

**트리거:** File A component 구현 대상 또는 File B 제품 페이지/frame(들)이 완성되고 target path가 정해졌을 때 **스킬:** `figma-implement` **입력:** File A component URL 또는 File B frame URL(들) with node-id, target path, `DESIGN.md`, `tailwind.theme.css`, `COMPONENTS.md`



## 실행 전 체크

- `DESIGN.md`, `COMPONENTS.md`가 있다.
- `tailwind.theme.css`가 앱 global stylesheet에 import되어 있다.
- Figma URL에 `node-id`가 있다.
- target component/route/file path가 있다.
- 반응형이면 breakpoint 값이 `DESIGN.md ## Layout` / `globals.css`에 있다.
- shadcn 컴포넌트가 필요하면 설치 여부를 확인했다.

## 개발자 -> 에이전트 (스킬 실행)

```
Figma URL(s): <File A component URL 또는 File B frame URL(들) with node-id>
Target: components/sections/PricingCard.tsx 또는 app/[route]/page.tsx
Use DESIGN.md, tailwind.theme.css, COMPONENTS.md.
Do not create new shared components unless existing ones cannot express the design.
(반응형) 프레임들을 mobile-first로 병합해줘: base=mobile, md:=tablet, lg:=desktop.
(shadcn) 설치된 건 재사용, 미설치 표준 shadcn은 CLI로 설치 후 등록, customized는 재생성 금지.
```

## 에이전트 처리 흐름

1. Figma MCP로 frame structure/assets 수집(반응형이면 모든 breakpoint 프레임).
2. `COMPONENTS.md` Lookup Order를 origin과 함께 따른다.
3. 기존 component를 props/variants/slots/composition으로 표현 가능한지 확인.
4. shadcn 노드 처리: 설치됨→재사용 / 미설치 표준→설치 후 등록 / customized→`do not regenerate` 준수.
5. component-level이면 matching entry를 `pending` →실제 path로 갱신.
6. page-level이면 등록된 component를 먼저 재사용.
7. 반응형이면 프레임들을 노드 매칭→diff(width/height/flex-direction)→mobile-first 병합.
8. 기존 component로 불가능할 때만 새 component 이유 기록 후 생성.
9. 구현 path, `COMPONENTS.md` 갱신 내역, 반응형 커버리지(병합한 breakpoint), blocker 보고.

## 완료 기준

- 구현된 style value가 `DESIGN.md` / `tailwind.theme.css` token으로 추적 가능(hardcoded 값 없음).
  - 반응형 출력은 문서화된 breakpoint에 묶인 접두사 유틸 사용(hardcoded px breakpoint 없음).
  - shadcn primitive는 재사용/CLI 설치만, hand-rewrite 없음. customized shadcn은 재생성 안 함.
  - shared component는 `COMPONENTS.md`에 있거나 생성 이유가 기록되어 있다.
-

# 스킬 요약

| 스킬                                | 입력                                                                                       | 출력                                                                                                                                                                              | 실행 빈도                           | 다음 단계                                                  |
|-----------------------------------|------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------|--------------------------------------------------------|
| <code>design.md</code>            | File 1 Foundations/Design System URL                                                     | <code>DESIGN.md</code> (+Layout breakpoint), <code>tailwind.theme.css</code> , <code>tokens.studio.global.json</code> , handoff <code>COMPONENTS.md</code> , shadcn token 매핑 보고 | 프로젝트 시작 및 token/breakpoint 변경 시 | <code>components</code>                                |
| <code>component-templates</code>  | File 1 component URL 또는 Components page URL                                              | <code>COMPONENTS.md</code> mapping contract (Origin 포함)                                                                                                                         | component 변경/shadcn 설치 시마다      | File 2 제품 페이지 디자인 또는 <code>figma-implementation</code> |
| <code>figma-implementation</code> | File A component URL 또는 File B frame URL(들) with node-id, target path, handoff artifacts | component-level 또는 page-level code (반응형 병합 포함)                                                                                                                                  | 컴포넌트/페이지 구현 시마다                 | 구현 검토 또는 Phase 2 loop                                  |

# 이름 일관성 규칙

```
Figma component: Button
COMPONENTS.md:    Button -> components/ui/button.tsx (Origin: shadcn)
Code file:        button.tsx
```

세 곳의 이름이 다르면 에이전트가 추측에 의존한다. shadcn Kit이면 Figma의 variant/size 이름을 shadcn props API와 정렬한다. 불일치 발견 시 `components` 스킬이 `Variants/Notes` 에 매핑 note를 남기고 정렬을 요청한다.

# 루프백 규칙

| 발견된 문제                                      | 돌아갈 phase | 이유                                                           |
|---------------------------------------------|-----------|--------------------------------------------------------------|
| token 부족 또는 Figma/code token 불일치            | Phase 1   | DESIGN.md 와 token artifacts 갱신 필요                            |
| breakpoint 값이 없거나 모호함                       | Phase 1   | DESIGN.md ## Layout 에 breakpoint 문서화 + globals.css wiring 필요 |
| Figma component mapping이 없거나 pending        | Phase 2   | COMPONENTS.md 갱신 또는 code component 구현/설치 필요                  |
| shadcn origin/custom 표기가 없음                 | Phase 2   | COMPONENTS.md 에 Origin 분류 필요                                 |
| 기존 component로 표현 불가능한 새 shared component 필요 | Phase 2   | 새 component 등록 및 재사용 contract 갱신 필요                          |
| 반응형인데 단일 프레임만 있음 / instance가 detach됨        | Phase 3   | breakpoint별 프레임을 같은 instance로 다시 정리 필요                       |
| 제품 page frame이 구현 불가능한 raw layout           | Phase 3   | library instance 기반으로 design 정리 필요                           |

## 운영 체크리스트

- design-md 실행 후 @google/design.md lint 가 통과했는가?
- tailwind.theme.css 가 fresh export와 일치하는가? (breakpoint/shadcn 변수가 섞여 들어가지 않았는가?)
- tokens.studio.global.json 이 wrapper 없는 token-set JSON인가?
- tailwind.theme.css 가 앱 stylesheet에 import되었는가?
- 커스텀 breakpoint가 globals.css 의 --breakpoint-\* 에 선언되었는가?
- shadcn 사용 시 --primary 등 변수가 token 값과 정렬되었는가?
- COMPONENTS.md 에 필요한 Figma component가 모두 있고 Origin이 표기되었는가?
- 커스텀 shadcn이 do not regenerate 로 표기되었는가?
- pending code path가 구현/설치 전에 정리되었는가?
- File 2 page frame은 File 1 library component instance를 사용했는가? (반응형이면 breakpoint별 프레임이 같은 instance인가?)
- figma-implement 후 구현 path, COMPONENTS.md 갱신 내역, 반응형 커버리지가 보고되었는가?